Homework 2 – P2: Hiểu và giải thích code

By **Nguyễn Sỹ Hùng** (Student ID: 25MSA33055), 2026-03-09

1. Introduction

Tài liệu này trình bày kết quả tìm hiểu và phân tích cách cài đặt thuật toán **Logistic Regression** thông qua 3 ví dụ code trong Google Colab mà Giảng viên đã cung cấp. Cụ thể, nội dung bao gồm: (1) "**from scratch**" theo phương pháp **non-vectorization**, (2) sử dụng thư viện **SKLearn**, và (3) phiên bản **vectorization** để tăng tốc tính toán.

Ngoài ra, tài liệu đưa ra **nhận xét từ kết quả thực nghiệm** nhằm làm rõ mức độ hiểu bài. Phần nhận xét đã trình bày đặc điểm của tập dữ liệu gồm các **samples có 2 features**, vì vậy có thể biểu diễn trong không gian **2D**, với hai dạng phân bố là **circle** và **moon**. Đồng thời, dữ liệu gồm **2 classes** khác nhau, được biểu diễn bằng các màu sắc khác nhau để dễ phân biệt, và được chia thành **training set** và **testing set**. Trên cơ sở đó, ta có báo cáo về phân tích cách mô hình học, cách đánh giá kết quả, và sự khác nhau giữa ba cách cài đặt.

2. Logistic Regression training from scratch

Code của phần này trên Colab là một implementation cho thuật toán Logistic Regression training from scratch theo cách **non-vectorization**, nghĩa là các phép tính được thực hiện bằng cách duyệt từng sample trong tập huấn luyện. Trong mỗi iteration, chương trình tính predicted probability, loss, các gradients của từng tham số, rồi cập nhật các tham số bằng gradient descent. Qua review và cách hiểu của mình, cho thấy rằng thuật toán được xây dựng đúng theo cơ sở toán học của Logistic Regression cho bài toán phân loại nhị phân.

Vui lòng xem các "comment" của mình cho phần code được trình bày bên dưới.

```
# 1. Tạo dữ liệu huấn luyện và kiểm tra

np.random.seed(10)

# Sinh ngẫu nhiên 2 cụm dữ liệu thuộc 2 lớp
X_1 = np.random.randn(1500, 2)            # lớp 0
X_2 = 1.2 * np.random.randn(1500, 2) - 4.5  # lớp 1

# Split train/test
X_1_train = X_1[:1000, :]
X_2_train = X_2[:1000, :]

X_1_test = X_1[1000:, :]
X_2_test = X_2[1000:, :]
```

```python
# Ghép dữ liệu train của hai lớp lại
X_train = np.vstack([X_1_train, X_2_train])   # shape = (2000, 2)

# Tạo vector label y cho train
# 1000 mẫu đầu là class 0, 1000 mẫu sau là class 1
y = np.zeros((2000, 1))
y[1000:, 0] = 1

# Ghép dữ liệu test
X_test = np.vstack([X_1_test, X_2_test])      # shape = (1000, 2)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test :", X_test.shape)
print("Shape of y      :", y.shape)


# 2. Hàm sigmoid
def sigmoid(z):
    # Chuyển z thành xác suất trong khoảng (0,1)
    return 1 / (1 + np.exp(-z))


# 3. Hàm loss cho 1 sample
def loss_function(y_true, a):
    """
    y_true : nhãn thật của 1 sample (0 hoặc 1)
    a      : xác suất dự đoán của model cho sample đó
    """
    # Cross-entropy loss cho logistic regression
    L = -(y_true * np.log(a) + (1 - y_true) * np.log(1 - a))
    return L


# 4. Tính gradient theo kiểu non-vectorization
def derivative_calculations(X, y_target, w1, w2, b):
    """
    Hàm này duyệt từng sample một để:
    - tính dự đoán
    - tính loss
    - cộng dồn gradient cho w1, w2, b

    Đây chính là cách tiếp cận non-vectorization - sẽ chậm hơn nhiều so với
vectorization, sẽ được phân tích phần sau.
    """

    # Khởi tạo gradient ban đầu bằng 0
    dw1 = 0
    dw2 = 0
    db = 0
    J = 0

    # m = số lượng mẫu trong training set
    m = X.shape[0]
```

```python
    # Duyệt từng sample
    for i in range(m):
        # Tính z của sample thứ i
        zi = w1 * X[i, 0] + w2 * X[i, 1] + b

        # Đưa qua sigmoid để ra xác suất dự đoán
        ai = sigmoid(zi)

        # Cộng loss của sample thứ i vào tổng cost
        J += loss_function(y_target[i, 0], ai)

        # Với logistic regression + cross-entropy:
        # dz = a - y
        dzi = ai - y_target[i, 0]

        # Gradient theo từng tham số
        dw1 += X[i, 0] * dzi
        dw2 += X[i, 1] * dzi
        db += dzi

    # Lấy trung bình trên toàn bộ m sample
    J /= m
    dw1 /= m
    dw2 /= m
    db /= m

    return dw1, dw2, db, J


# 5. Hàm này thực hiện training sử dụng gradient descent
def iteration_step(X, y_target, num_it, alpha, w1, w2, b):
    """
    num_it : số lần lặp cập nhật tham số
    alpha  : learning rate

    Mỗi iteration:
    1. Tính gradient trên toàn bộ tập train
    2. Cập nhật w1, w2, b
    3. Lưu lại lịch sử tham số và cost
    """

    # Lưu lịch sử để quan sát quá trình học
    w1_list = []
    w2_list = []
    b_list = []
    J_list = []

    for i in range(num_it):
        # Tính gradient và cost tại bộ tham số hiện tại
        dw1, dw2, db, J_cost = derivative_calculations(X, y_target, w1, w2, b)

        # Cập nhật tham số theo gradient descent
        w1 = w1 - alpha * dw1
        w2 = w2 - alpha * dw2
```

```
        b = b - alpha * db

        # Lưu lại để theo dõi
        w1_list.append(w1)
        w2_list.append(w2)
        b_list.append(b)
        J_list.append(J_cost)

    return w1_list, w2_list, b_list, J_list


# 6. Thực thi việc training

n_iterations = 100000

# Khởi tạo tham số ban đầu bằng 0
w1_init = 0
w2_init = 0
b_init = 0

w1_list, w2_list, b_list, J_list = iteration_step(
    X_train, y,
    num_it=n_iterations,
    alpha=0.1,
    w1=w1_init,
    w2=w2_init,
    b=b_init
)

# Lấy tham số cuối cùng sau huấn luyện
w1_final = w1_list[-1]
w2_final = w2_list[-1]
b_final = b_list[-1]

print("w1_final =", w1_final)
print("w2_final =", w2_final)
print("b_final  =", b_final)


# 7. Visualize quá trình hội tụ

plt.plot(J_list, label='Cost J')
plt.plot(w1_list, label='w1')
plt.plot(w2_list, label='w2')
plt.plot(b_list, label='b')

plt.xlabel('Iteration')
plt.title('Quá trình thay đổi cost và parameters')
plt.legend()
plt.grid()
plt.show()
```

```python
# 8. Hàm dự đoán cho từng sample
def predict_non_vectorized(X, w1, w2, b):
    """
    Dự đoán nhãn 0/1 cho từng sample trong X
    """
    m = X.shape[0]
    y_pred = np.zeros((m, 1))

    for i in range(m):
        zi = w1 * X[i, 0] + w2 * X[i, 1] + b
        ai = sigmoid(zi)

        # Ngưỡng phân loại 0.5
        if ai > 0.5:
            y_pred[i, 0] = 1
        else:
            y_pred[i, 0] = 0

    return y_pred

# 9. Tính accuracy trên train
y_pred_train = predict_non_vectorized(X_train, w1_final, w2_final, b_final)

train_acc = 100 - np.mean(np.abs(y_pred_train - y)) * 100
print("Train accuracy: {:.2f}%".format(train_acc))
```

3. Logistic Regression training from SKLearn

Notebook này vẫn tiếp tục với toán binary classification như trước. Tuy nhiên lần này, code dùng LogisticRegression của SKLearn để huấn luyện mô hình trên X_train và y để học một **linear decision boundary** giữa hai classes. Kết quả dự đoán trên **test set** được kiểm tra bằng **confusion matrix** và bằng cách quan sát các điểm bị **misclassified** trên đồ thị. Cuối cùng, code so sánh đường phân tách của mô hình **"From scratch"** và mô hình **SKLearn**, cho thấy hai cách tiếp cận cho kết quả gần tương tự nhau.

Dưới đây là các comment cho code:

```python
# 1. Tạo dữ liệu cho 2 classes, mỗi sample có 2 features

# Class 0: sinh ngẫu nhiên quanh gốc tọa độ
X_1 = np.random.randn(1500, 2)

# Class 1: sinh ngẫu nhiên rồi scale và shift để tạo cụm khác biệt
X_2 = 1.2 * np.random.randn(1500, 2) - 4.5

# 2. Split dataset thành training set và test set
# Lấy 1000 sample đầu của mỗi class làm train
X_1_train = X_1[:1000, :]
X_2_train = X_2[:1000, :]
```

```
# Lấy 500 sample còn lại của mỗi class làm test
X_1_test = X_1[1000:, :]
X_2_test = X_2[1000:, :]

# Ghép dữ liệu train của 2 class lại thành một ma trận X_train
X_train = np.vstack([X_1_train, X_2_train])

# Tạo vector nhãn y cho training set
# 1000 sample đầu là class 0
# 1000 sample sau là class 1
y = np.zeros((2000, 1))
y[1000:, 0] = 1

# Ghép dữ liệu test của 2 class lại thành X_test
X_test = np.vstack([X_1_test, X_2_test])

# Tạo nhãn thật cho test set để dùng khi đánh giá
# 500 sample đầu là class 0, 500 sample sau là class 1
y_test = np.zeros((1000, 1))
y_test[500:, 0] = 1

print('Shape of the training set is (%i, %i).' % X_train.shape)
print('Shape of the test set is (%i, %i).' % X_test.shape)
print('Shape of the target vector is (%i, %i).' % y.shape)

# 3. Plot training set và test set để quan sát dữ liệu
plt.scatter(X_1_train[:, 0], X_1_train[:, 1], label='class 0 train',
color='orange')
plt.scatter(X_2_train[:, 0], X_2_train[:, 1], label='class 1 train', color='blue')

plt.scatter(X_1_test[:, 0], X_1_test[:, 1], label='class 0 test', color='red')
plt.scatter(X_2_test[:, 0], X_2_test[:, 1], label='class 1 test', color='green')

plt.legend()
plt.show()

# 4. Khởi tạo và huấn luyện dùng Logistic Regression của SKLearn

# Tạo model Logistic Regression với các tham số mặc định
LR = LogisticRegression()

# Train model
# SKLearn thích y có shape (n_samples,) hơn là (n_samples, 1)
LR.fit(X_train, y.reshape(y.shape[0], ))

# Lấy ra trọng số (weights) và bias đã học được
print('Coefficients of LogisticRegression from sklearn are in (%i, %i) dimensional
matrix'
      % LR.coef_.shape)

w1_skl = LR.coef_[0, 0]
w2_skl = LR.coef_[0, 1]
b_skl = LR.intercept_[0]
```

```
print('w1 = %f' % w1_skl)
print('w2 = %f' % w2_skl)
print('b = %f' % b_skl)

# 5. Dự đoán trên test set

# Predict class label cho từng sample trong X_test
y_test_sklearn = LR.predict(X_test).reshape(1000, 1)

# 6. Tính confusion matrix để đánh giá kết quả phân loại

cm_sk = metrics.confusion_matrix(y_test, y_test_sklearn)
print("Confusion matrix:")
print(cm_sk)

# 7. Visualize test set trước và sau khi classification

plt.figure(figsize=(10, 10))

# Hình 1: dữ liệu test gốc
plt.subplot(2, 1, 1)
plt.scatter(X_1_test[:, 0], X_1_test[:, 1], label='class 0', color='red')
plt.scatter(X_2_test[:, 0], X_2_test[:, 1], label='class 1', color='green')
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.title('Original dataset, before classification')
plt.legend()

# Hình 2: dữ liệu test sau khi model classify
plt.subplot(2, 1, 2)

# 500 sample đầu thuộc class 0 thật
for i in range(0, 500):
    # Vẽ màu đỏ vì ground truth là class 0
    plt.scatter(X_test[i, 0], X_test[i, 1], color='red')

    # Nếu model dự đoán thành class 1 thì đây là misclassified
    if y_test_sklearn[i] == 1:
        plt.scatter(X_test[i, 0], X_test[i, 1], color='green')
        print('Element from class 0 with index %i, feature1 = %f and feature2 = %f
is misclassified.'
              % (i, X_test[i, 0], X_test[i, 1]))

# 500 sample sau thuộc class 1 thật
for i in range(500, 1000):
    # Vẽ màu xanh lá vì ground truth là class 1
    plt.scatter(X_test[i, 0], X_test[i, 1], color='green')

    # Nếu model dự đoán thành class 0 thì đây là misclassified
    if y_test_sklearn[i] == 0:
        plt.scatter(X_test[i, 0], X_test[i, 1], color='red')
        print('Element from class 1 with index %i, feature1 = %f and feature2 = %f
is misclassified.'
              % (i, X_test[i, 0], X_test[i, 1]))
```

```
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.title('Dataset when it is classified with sklearn LogisticRegression')

# Khoanh tròn một vài điểm bị phân loại sai để minh họa
circle1 = plt.Circle((-1.879757, -2.862382), radius=0.22, color='red', fill=False)
circle2 = plt.Circle((-2.613556, -1.879211), radius=0.20, color='red', fill=False)
circle3 = plt.Circle((-0.667955, -2.092435), radius=0.25, color='green',
fill=False)

plt.text(-4, -0.8, 'These elements')
plt.text(-4, -1.2, 'are misclassified')

plt.gca().add_patch(circle1)
plt.gca().add_patch(circle2)
plt.gca().add_patch(circle3)

plt.show()

# 8. So sánh với model tạo ra ở notebook "Logistic Regression from scratch"

# Đây là tham số của model Logistic Regression được train from scratch
w1_final = -2.882705
w2_final = -3.150111
b_final = -12.158909

# Tạo trục x để vẽ decision boundary
xx = np.linspace(-6, 2)

# Decision boundary có dạng:
# w1*x1 + w2*x2 + b = 0
# => x2 = -(w1/w2)*x1 - b/w2
yy_lr = -(w1_final / w2_final) * xx - b_final / w2_final
yy_sk = -(w1_skl / w2_skl) * xx - b_skl / w2_skl

# So sánh trên test set
plt.figure(figsize=(10, 7))
plt.scatter(X_1_test[:, 0], X_1_test[:, 1], label='class 0', color='red')
plt.scatter(X_2_test[:, 0], X_2_test[:, 1], label='class 1', color='green')
plt.plot(xx, yy_lr, label='Logistic Regression from scratch', color='green')
plt.plot(xx, yy_sk, label='Logistic Regression from sklearn', color='black')
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.title('Comparison of Logistic Regression from scratch and sklearn on test
set')
plt.legend()
plt.grid()
plt.show()

# So sánh trên training set
plt.figure(figsize=(10, 7))
plt.scatter(X_1_train[:, 0], X_1_train[:, 1], label='class 0', color='orange')
plt.scatter(X_2_train[:, 0], X_2_train[:, 1], label='class 1', color='blue')
```

```
plt.plot(xx, yy_lr, label='Logistic Regression from scratch', color='green')
plt.plot(xx, yy_sk, label='Logistic Regression from sklearn', color='black')
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.title('Comparison of Logistic Regression from scratch and sklearn on train
set')
plt.legend()
plt.grid()
plt.show()
```

4. Logistic Regression training fast using vectorization

Ở notebook này, vẫn là bài toán như trong Notebook đầu, tuy nhiên khác là nó áp dụng cách tiếp cận vectorization thay vì non- vectorization. Nói cách khác, nó là phiên bản vectorization của "Logistic Regression from scratch". vectorization nghĩa là tính toán đồng thời trên toàn bộ tập dữ liệu bằng **matrix operations** thay vì xử lý từng sample riêng lẻ vì thế sẽ cho tốc độ nhanh hơn nhiều.

Dưới đây là các comment cho code:

```
# 1. Generate dataset
# Tạo dữ liệu mẫu cho binary classification
# Có thể chọn make_circles hoặc make_moons
#X, y = make_circles(n_samples=3000, noise=0.08, factor=0.3)
X, y = make_moons(n_samples=3000, noise=0.08)

# Kiểm tra shape và kiểu dữ liệu
print(X.shape)
print(y.shape)
print(type(X))
print(type(y))

# 2. Tách dữ liệu theo từng class

# X0 sẽ chứa các sample thuộc class 0
# X1 sẽ chứa các sample thuộc class 1
X0 = []
X1 = []

for i in range(len(y)):
    if y[i] == 0:
        X0.append(X[i, :])
    else:
        X1.append(X[i, :])

# Chuyển từ list sang numpy array để dễ xử lý vectorized
X0_np = np.array(X0)
X1_np = np.array(X1)

# 3. Chia thành training set và test set
```

```
# Ở đây dùng transpose để đưa dữ liệu về dạng: mỗi cột là 1 sample, mỗi hàng là 1
feature để có thể áp dụng phép toán ma trận
X0_train = X0_np[:1000, :].T
X0_test  = X0_np[1000:, :].T

X1_train = X1_np[:1000, :].T
X1_test  = X1_np[1000:, :].T

# Ghép training samples của 2 class lại
X_train = np.hstack([X0_train, X1_train])

# Tạo nhãn cho training set
# 1000 sample đầu thuộc class 0
# 1000 sample sau thuộc class 1
y_train = np.zeros((1, 2000))
y_train[0, 1000:] = 1

# Ghép test samples của 2 class lại
X_test = np.hstack([X0_test, X1_test])

# Tạo nhãn cho test set
# 500 sample đầu thuộc class 0
# 500 sample sau thuộc class 1
y_test = np.zeros((1, 1000))
y_test[0, 500:] = 1

# 4. Visualize dữ liệu để quan sát distribution

plt.scatter(X0_train[0, :], X0_train[1, :], color='b', label='class 0 train')
plt.scatter(X1_train[0, :], X1_train[1, :], color='r', label='class 1 train')
plt.scatter(X0_test[0, :], X0_test[1, :], color='LightBlue', label='class 0 test')
plt.scatter(X1_test[0, :], X1_test[1, :], color='Orange', label='class 1 test')

plt.xlabel('feature1')
plt.ylabel('feature2')
plt.legend()
plt.axis('equal')
plt.show()

# In shape để kiểm tra dữ liệu đã được tổ chức đúng chưa
print('Shape of X_train set is %i x %i.' % X_train.shape)
print('Shape of X_test set is %i x %i.' % X_test.shape)
print('Shape of y_train set is %i x %i.' % y_train.shape)
print('Shape of y_test set is %i x %i.' % y_test.shape)

# 5. Định nghĩa hàm sigmoid
def sigmoid(z):
    # sigmoid biến giá trị bất kỳ thành xác suất trong khoảng (0,1)
    s = 1 / (1 + np.exp(-z))
    return s

# 6. Forward + cost + gradients cho single neuron
def single_neuron(w, b, X, Y):
    # m là số lượng samples
```

```
    m = X.shape[1]

    # Forward propagation cho toàn bộ tập dữ liệu cùng lúc
    # w.T có shape (1, n_features)
    # X có shape (n_features, m)
    # nên np.dot(w.T, X) cho ra shape (1, m)
    A = sigmoid(np.dot(w.T, X) + b)

    # Tính cost theo binary cross-entropy
    cost = (-1 / m) * np.sum(Y * np.log(A) + (1 - Y) * np.log(1 - A))

    # Backward propagation theo dạng vectorized
    dw = (1 / m) * np.dot(X, (A - Y).T)
    db = (1 / m) * np.sum(A - Y)

    # Lưu gradients vào dictionary
    grads = {
        "dw": dw,
        "db": db
    }

    return grads, cost

# 7. Tối ưu tham số bằng gradient descent
def optimize(w, b, X, Y, num_iterations, learning_rate):
    costs = []

    for i in range(num_iterations):
        # Mỗi vòng lặp:
        # - tính cost hiện tại
        # - tính gradients hiện tại
        grads, cost = single_neuron(w, b, X, Y)

        # Lấy đạo hàm ra từ dictionary
        dw = grads["dw"]
        db = grads["db"]

        # Cập nhật parameters theo gradient descent
        w = w - learning_rate * dw
        b = b - learning_rate * db

        # Lưu lại cost để vẽ learning curve
        costs.append(cost)

    # Sau khi train xong, lưu parameters cuối cùng
    params = {
        "w": w,
        "b": b
    }

    return params, costs

# 8. Khởi tạo parameters ban đầu
# w có shape (n_features, 1)
```

```python
# nhân 0.01 để giá trị khởi tạo nhỏ
w = np.random.randn(X_train.shape[0], 1) * 0.01
b = 0

# Train model
param_w_b, costs = optimize(
    w, b,
    X_train, y_train,
    num_iterations=10000,
    learning_rate=0.09
)

# Visualize cost theo số vòng lặp để xem quá trình học
plt.plot(costs)
plt.xlabel('number of iterations')
plt.title('Cost function')
plt.show()

# Kiểm tra shape của parameters sau khi train
print(param_w_b['w'].shape)
print(type(param_w_b['b']))

# 9. Hàm dự đoán
def make_predictions(w, b, X):
    # m là số lượng samples cần dự đoán
    m = X.shape[1]

    # Tạo ma trận prediction ban đầu
    Y_predictions = np.zeros((1, m))

    # Tính xác suất dự đoán cho toàn bộ samples cùng lúc
    A = sigmoid(np.dot(w.T, X) + b)

    # Biến xác suất thành class label
    # Nếu > 0.5 thì gán class 1, ngược lại là class 0
    for i in range(A.shape[1]):
        Y_predictions[0, i] = 1 if A[0, i] > 0.5 else 0

    # Kiểm tra shape để tránh lỗi dimension
    assert(Y_predictions.shape == (1, m))

    return Y_predictions

# 10. Dự đoán trên training set và test set
Y_predictions_train = make_predictions(param_w_b['w'], param_w_b['b'], X_train)
Y_predictions_test  = make_predictions(param_w_b['w'], param_w_b['b'], X_test)

# 11. Visualize kết quả trên training set

plt.subplot(1, 2, 1)
plt.scatter(X0_train[0, :], X0_train[1, :], color='Blue', label='class 0 train')
plt.scatter(X1_train[0, :], X1_train[1, :], color='Red', label='class 1 train')
plt.xlabel('feature1')
plt.ylabel('feature2')
```

```
plt.title('Training set')
plt.axis('equal')

plt.subplot(1, 2, 2)
plt.scatter(X0_train[0, :], X0_train[1, :], color='Blue', label='class 0 train')
plt.scatter(X1_train[0, :], X1_train[1, :], color='Red', label='class 1 train')

for i in range(0, 1000):
    # Đây là class 0 thật, nếu predict thành 1 thì sai
    if Y_predictions_train[0, i] == 1:
        plt.scatter(X_train[0, i], X_train[1, i], color='Red')

for i in range(1000, 2000):
    # Đây là class 1 thật, nếu predict thành 0 thì sai
    if Y_predictions_train[0, i] == 0:
        plt.scatter(X_train[0, i], X_train[1, i], color='Blue')

plt.xlabel('feature1')
plt.title('Classified')
plt.axis('equal')
plt.show()

# Tính accuracy trên training set
print("train accuracy: {} %".format(
    100 - np.mean(np.abs(Y_predictions_train - y_train)) * 100
))

# 12. Visualize hóa kết quả trên test set
plt.subplot(1, 2, 1)
plt.scatter(X0_test[0, :], X0_test[1, :], color='LightBlue', label='class 0 test')
plt.scatter(X1_test[0, :], X1_test[1, :], color='Orange', label='class 1 test')
plt.ylabel('feature2')
plt.xlabel('feature1')
plt.title('Test set')
plt.axis('equal')

plt.subplot(1, 2, 2)
plt.scatter(X0_test[0, :], X0_test[1, :], color='LightBlue', label='class 0 test')
plt.scatter(X1_test[0, :], X1_test[1, :], color='Orange', label='class 1 test')

for i in range(0, 500):
    # Đây là class 0 thật, nếu predict thành 1 thì sai
    if Y_predictions_test[0, i] == 1:
        plt.scatter(X_test[0, i], X_test[1, i], color='Orange')

for i in range(500, 1000):
    # Đây là class 1 thật, nếu predict thành 0 thì sai
    if Y_predictions_test[0, i] == 0:
        plt.scatter(X_test[0, i], X_test[1, i], color='LightBlue')

plt.xlabel('feature1')
plt.axis('equal')
plt.title('Classified')
plt.show()
```

```
# Tính accuracy trên test set
print("test accuracy: {} %".format(
    100 - np.mean(np.abs(Y_predictions_test - y_test)) * 100
))
```

Phần training là chỗ chính áp dụng vectorization và nó giúp tăng tốc độ lên rất nhiều.

Bên cạnh đó, ở code gốc, mình thấy hàm ***make_predictions***() dung một vòng ***for*** để threshold từng phần tử và thực tế cũng có thể áp dụng **vectorization** như sau:

```
def make_predictions(w, b, X):
    # Tính xác suất cho toàn bộ samples cùng lúc
    A = sigmoid(np.dot(w.T, X) + b)

    # Threshold toàn bộ vector cùng lúc:
    # True -> 1, False -> 0
    Y_predictions = np.where(A > 0.5, 1, 0)

    return Y_predictions
```

5. Conclusion

Ba assignments cho thầy cùng một bài toán có thể implement theo ít nhất 3 cách khác nhau: non-vectorization, SKLearn, và vectorization. Qua đó, giúp mình thấy rõ hơn mô hình học từ dữ liệu như thế nào, cách dự đoán binary classification, và vai trò của weights, bias, loss/cost, cùng decision boundary. Tóm lại, SKLearn giúp giảm thời gian và công sức viết code, còn vectorization giúp chương trình chạy nhanh hơn.

Thank you!

Nguyễn Sỹ Hùng, Mar 2026